Introduction to Backend Developemnt using Node.js Part1

By Wafa Adham

Frontend vs Backend

- Frontend runs on the client-side inside the web browser and focuses on user interface (UI) and user experience (UX).
- Backend runs on the server-side and is responsible for processing data, executing business logic, and managing databases.
- Frontend technologies include HTML, CSS, and JavaScript.
- Backend communicates with the frontend using HTTP requests and APIs in a client-server architecture.

Frontend vs Backend

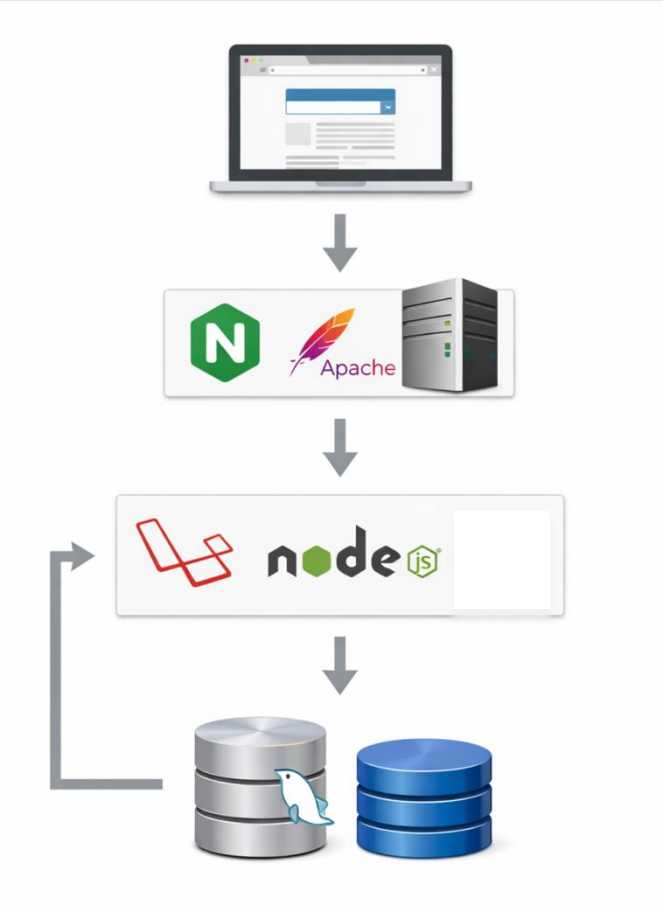
Backend

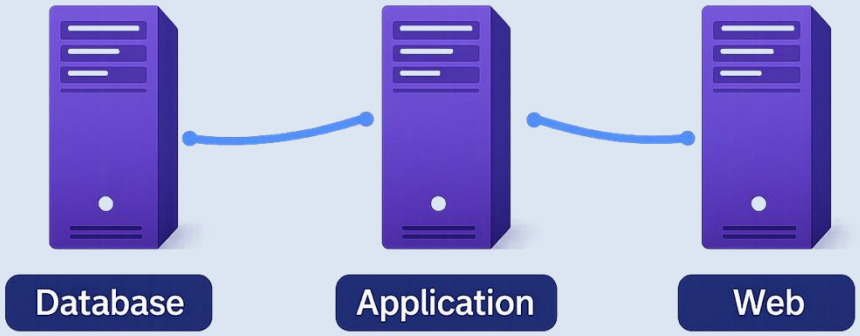
- Backend development refers to building the **server-side logic** of web and cloud applications.
- It includes managing servers, databases, application logic, and APIs.
- Backend ensures data is stored, processed, and delivered correctly.
- It acts as the core engine that powers the application behind the scenes.

Types of Servers

- **Web Server** – Handles HTTP requests and delivers web content to clients.
- **Application Server** – Executes business logic and processes dynamic content.
- **Database Server** – Stores and manages structured data.
- These servers work together to support client-server applications.

Types of Servers





MySql
MongoDB

Laravel
Nodejs

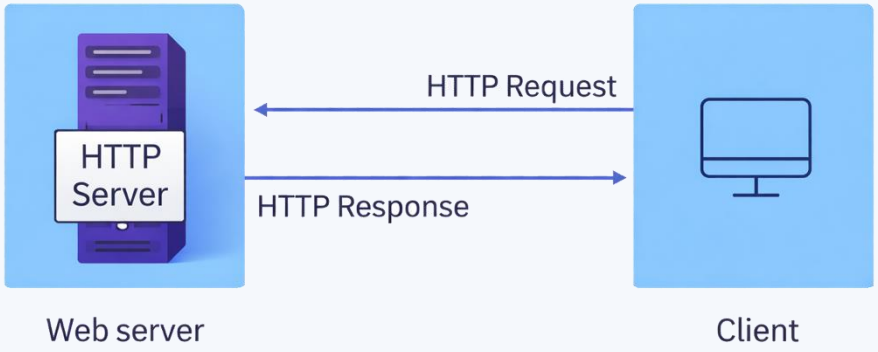
Apache
Nginx

Database Servers

- Responsible for storing, retrieving, and managing application data.
- Ensure data persistence and consistency.
- Examples include MySQL, PostgreSQL, SQL Server, and MongoDB.
- Often referred to simply as 'databases' in development discussions.

Web & HTTP Servers

- Respond to client requests using the Hypertext Transfer Protocol (HTTP).
- Serve static files such as HTML, CSS, images, and JavaScript.
- Control how users access hosted resources.
- Examples include Apache, Nginx, and Microsoft IIS.



Application Servers

- Host and execute business logic of the application.
- Process data received from web servers.
- Interact with database servers to fetch or store information.
- Generate dynamic content before sending it back to the client.

Backend Languages

- Used to implement server-side functionality and logic.
- Common languages: JavaScript (Node.js), Python, Java, PHP, Ruby, C#.
- Handle data processing, authentication, and communication with databases.
- Choice of language depends on scalability, ecosystem, and project needs.

Frameworks

- **Provide structure** and reusable components for backend development.
- Help developers follow best practices and speed up development.
- Examples: Django (Python), Ruby on Rails (Ruby), Express.js (JavaScript).
- Reduce boilerplate code and improve maintainability.

Runtime Environments

- Environment where backend code is executed.
- Provides infrastructure and system resources for applications to run.
- Acts like a **mini operating system** for the application.
- Example: **Node.js runtime** for executing JavaScript on the server.

Node.js

- **JavaScript runtime** built on Google Chrome's V8 engine.
- Allows developers to use JavaScript on both frontend and backend.
- **Event-driven** and **non-blocking** architecture improves performance.
- Popular for building scalable network applications and APIs.

Backend Responsibility (Scalability)

- Scalability is the ability of an application to handle growing user demand.
- Load includes number of users, transactions, and data transfer volume.
- Backend must dynamically manage resources without degrading performance.
- Backend is also responsible for security, authentication, and reliability.

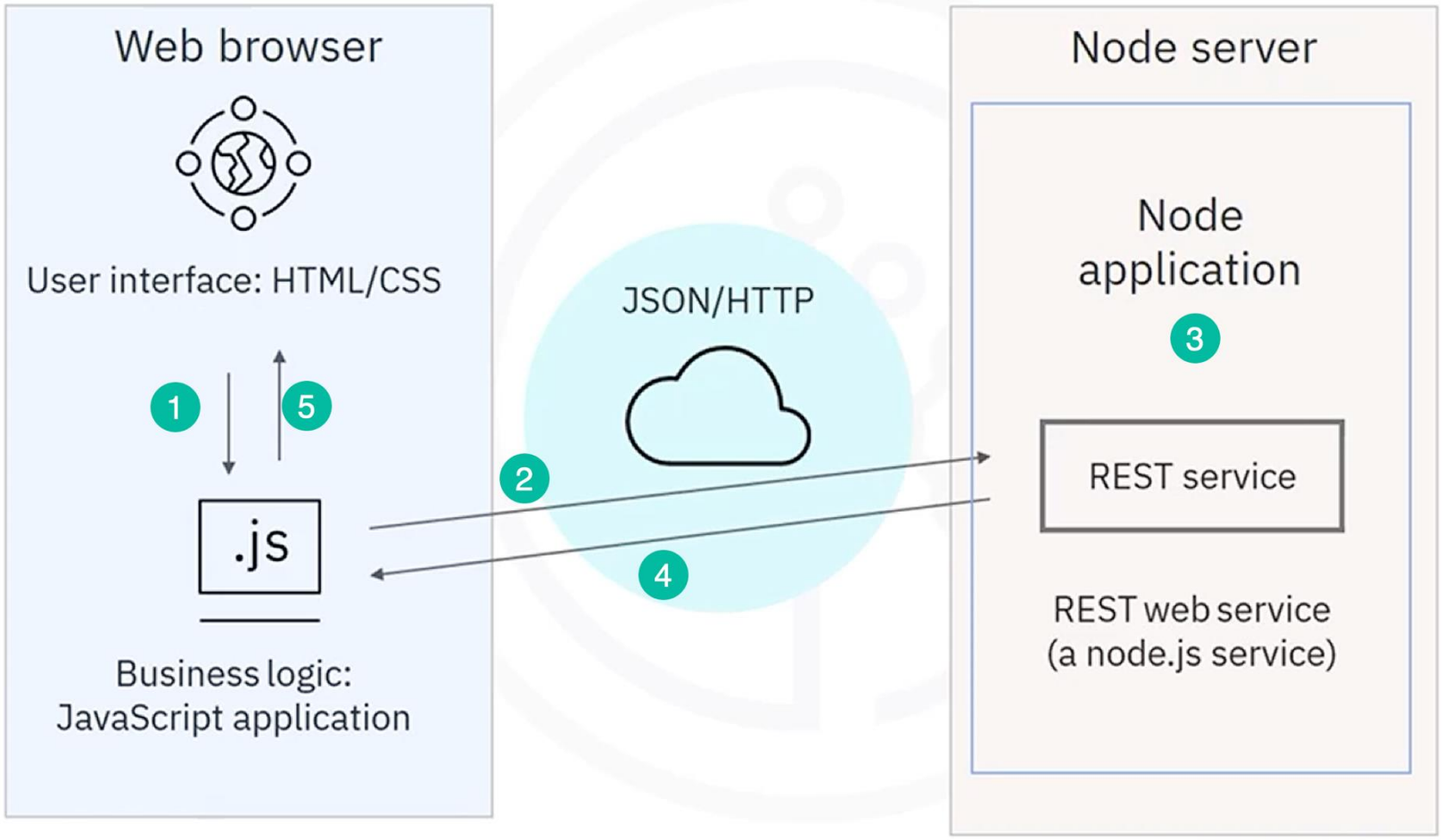
Intro to Node.js

- Node.js is an open-source runtime environment built on Google Chrome’s V8 engine.
- It enables JavaScript to run on the server-side.
- Node.js is event-driven and uses asynchronous, non-blocking I/O.
- It improves performance and scalability for web applications.

JavaScript with Node.js

- JavaScript is traditionally used for client-side scripting in browsers.
- Node.js allows JavaScript to be used on the server-side.
- Client-side JS handles UI logic (e.g., validation).
- Server-side Node.js processes HTTP requests and returns JSON responses.

JavaScript with Node.js - cont



Node.js Usage

- Processes and routes web service requests from clients.
- Handles REST APIs and JSON payloads over HTTP.
- Used in production by companies like Uber, LinkedIn, PayPal, and eBay.
- Can replace backend technologies such as Java, Python, Ruby, and C++ in many cases.

Express.js

- **Express.js** is a flexible framework **built on top of Node.js.**
- Simplifies building web applications and APIs.
- Provides routing, middleware, and HTTP utility methods.
- Abstracts low-level Node.js APIs to speed up development.

Javascript Modules

- A module is a file containing related and encapsulated JavaScript code.

- Modules serve a specific purpose within an application.
- They can be a single file or a collection of files (package).
- Modules improve reusability and maintainability by breaking complex code into manageable parts.

Package Specifications

- A package is a directory containing bundled modules.
- Module specifications define standards for creating and organizing packages.

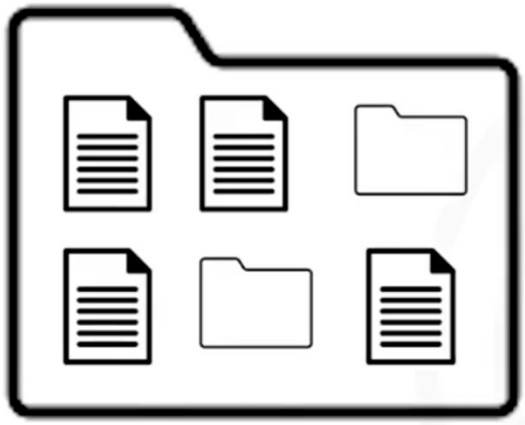
- CommonJS and ES Modules are the main specifications in Node.js.

- Node.js treats JavaScript as CommonJS by default.

Why Using import() and require()?

- Used to include external modules into an application.
- Allow applications to reuse functionality from other files.
- Help structure large-scale applications into logical components.
- Choice between them depends on module specification (CommonJS or ES Modules).

Package Specifications - cont



Package: a directory with one or more modules

Module specifications:

- Conventions and standards used to create packages
- CommonJS
- ES

Working with ES Modules

- Enabled by using .mjs file extension or configuration.
- Use the 'export' keyword to export variables or functions.
- Use the 'import' statement to include modules.
- Provides static binding and better optimization for large applications.

Importing and Exporting Module

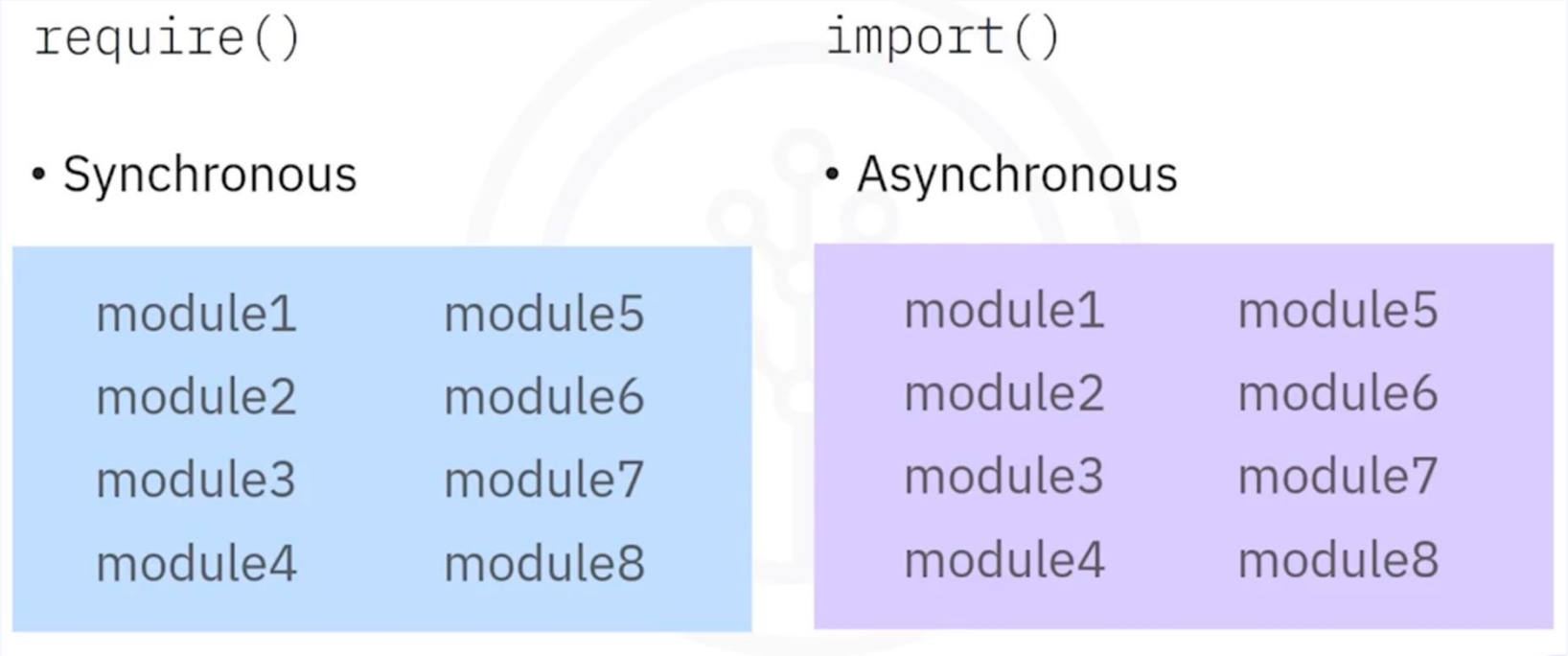
- CommonJS Export: module.exports = value;
- CommonJS Import: const data = require('./file');
- ES Export: export { variableName };
- ES Import: import { variableName } from './file.mjs';
- Modules must be exported before they can be imported.

	CommonJS	ES
Import a module	`require()`	`import()`
Export from a module	`module.exports`	`export`

Comparison between import and require

`require()`	`import()`
• Can be called anywhere in the code • Can be called within conditionals and functions • Dynamic • Binding errors not identified until run-time	• Can only be called at the beginning of the file • Cannot be called within conditionals or functions • Static • Binding errors identified at compile-time

Synchronous vs Asynchronous



- require() is synchronous (loads modules one at a time).
- import is asynchronous (can load modules simultaneously).
- Synchronous execution blocks further processing until complete.
- Asynchronous execution improves performance and scalability.

Import / require (Sample Code)

require()

```
//export from a file named
//message.js
module.exports = 'Hello
  Programmers';
```

```
//import from the message.js
//file
let msg =
  require('./message.js');
console.log(msg);
```

import()

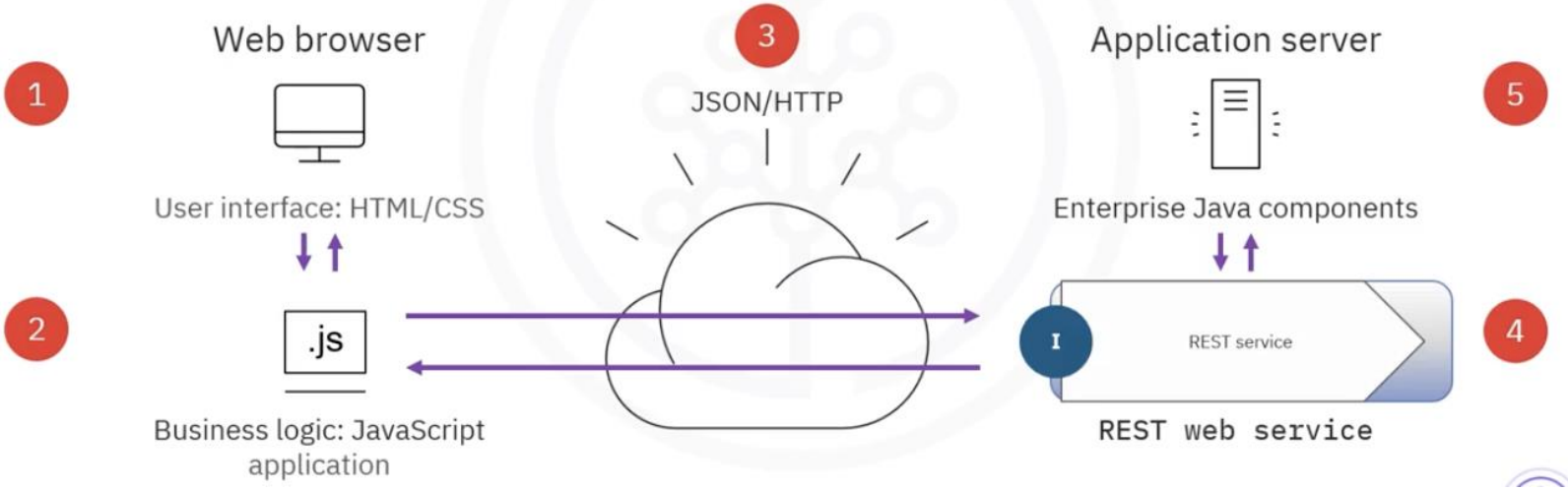
```
//export from file named module.mjs
const a = 1;
export { a as "myvalue" };
```

```
//import from module.mjs
import { myvalue } from
  module.mjs";
```

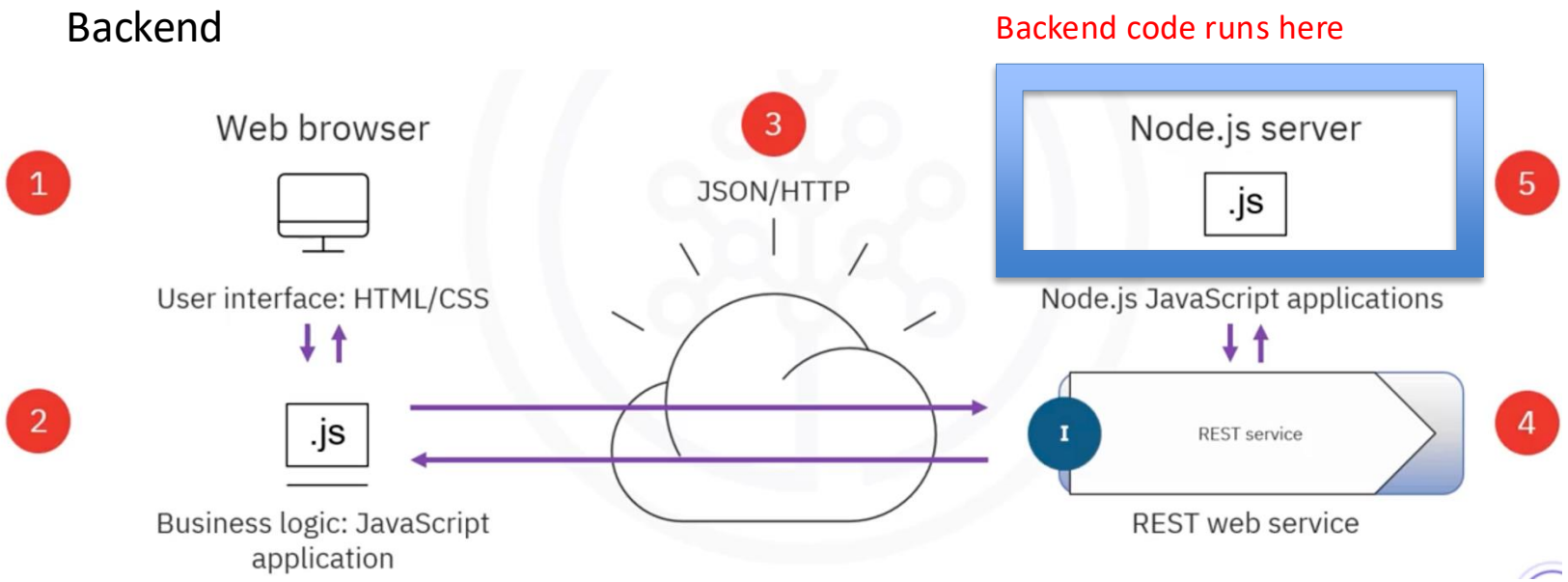
Frontend vs Backend

Frontend

Frontend code runs here



Frontend vs Backend



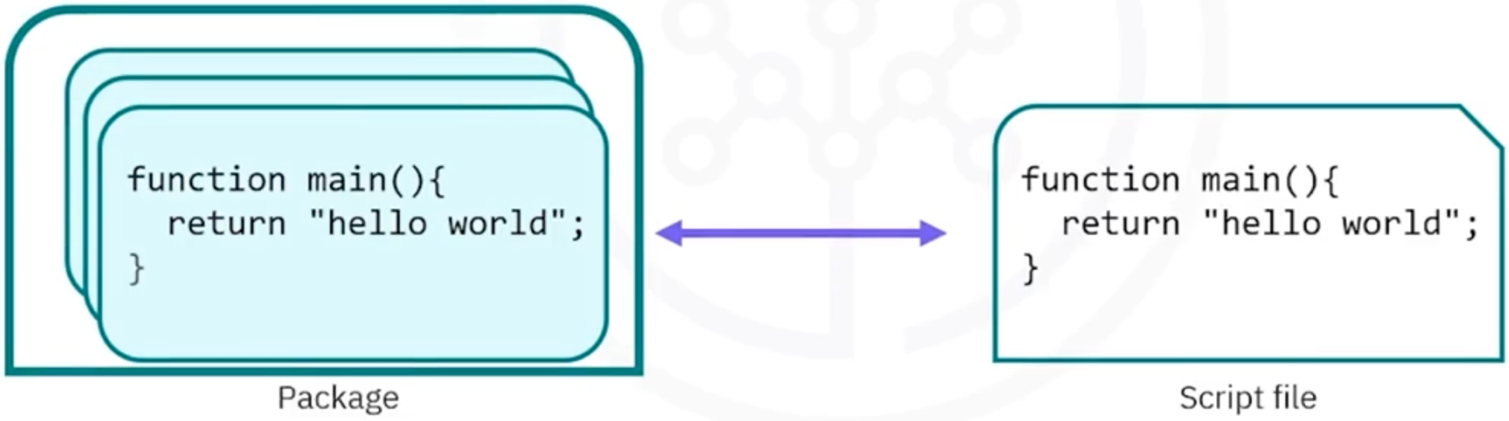
Nodejs : server side javascript

- Heavy Emphasis on concurrent programming
- Single threaded operations that handles I/O using events .
- Callback functions are used to handle results when they are complete.
- Used to build “scalable” web applications and support concurrency.

- Every javascript file ←→ Module

Nodejs : server side javascript

- Multiple node modules can be grouped into a package.



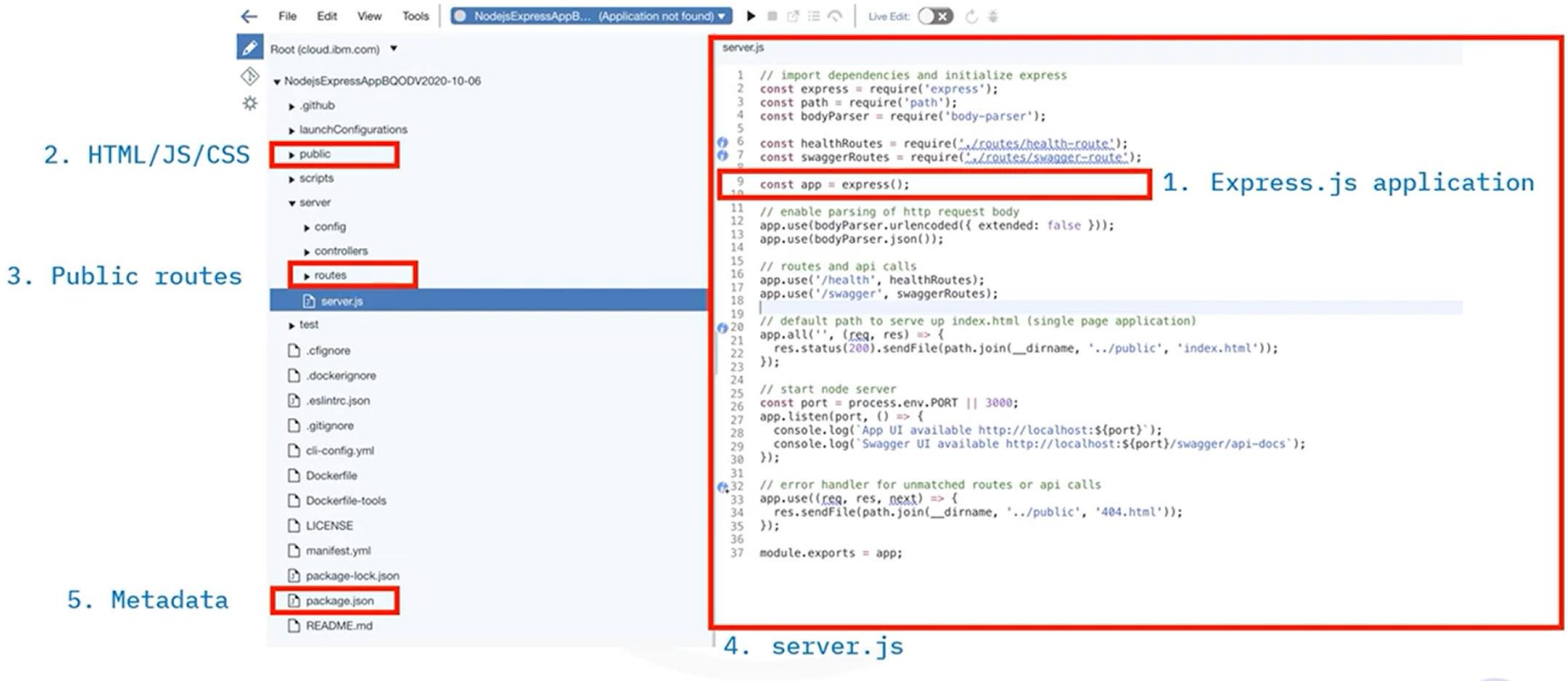
Simple http webserver using Nodejs

```
let server = http.createServer(function(request, response) {

 let body = "Hello world!";

 response.writeHead( 200, {

  'Content-Length': body.length,

  'Content-Type': 'text/plain'

 });

 response.end(body);

});

server.listen(8080);
```

Node and Express in IDE

- Server.js – main application entry point.
- Package.json – project metadata and dependencies.
- Routes – define API endpoints for handling requests.
- Views/Templates – server-rendered HTML responses.
- Public folder – static assets like CSS, images, and JavaScript files.

Node and Express in IDE - cont



Semantic Versioning 2.1.0-RC

Given a version number **MAJOR . MINOR . PATCH**, increment the:

1. MAJOR : version when you make incompatible API changes
2. MINOR : version when you add functionality in a backward compatible manner
3. PATCH : version when you make backward compatible bug fixes

- Additional labels for **pre-release** and build metadata are available as extensions to the MAJOR.MINOR.PATCH format.
- dev, alpha, beta, RC, and stable where RC stands for release candidate.

version constrains

1- Exact Version Constraint

Example: 1.0.2

2- Version Range
by using >, >=, <, <=, !=

>=1.0

>=1.0 <2.0

>=1.0 <1.1 || >=1.2

3- Hyphenated Version Range (-)

`1.0 - 2.0`
is equivalent to >=1.0.0 < 2.1 as
the 2.0 becomes 2.0.*

`1.0.0 - 2.1.0`
is equivalent to >=1.0.0 <=2.1.0

4- Wildcard Version Range (.*)
1.0.* is the equivalent of
>=1.0 <1.1.

version constrains –examples1

5- Tilde Version Range (~)
Include everything greater than a particular version **in the same minor range**

~ allows updates only within the same MINOR version (patch updates only).

~1.2 is equivalent to >=1.2 <2.0.0

`~1.2.3`
is equivalent to >=1.2.3 < 1.3.0

Another way of looking at it is that
using ~ specifies a minimum version, but allows the last digit specified to go up.

version constrains –examples2

- ~1.2
- >=1.2.0 <1.3.0

Allowed

1.2.0

1.2.1

1.2.5

1.2.99

1.3.0 ✗

- ~1

>=1.0.0 <2.0.0

~0.5.2

>=0.5.2 <0.6.0

- ~0.0.4

>=0.0.4 <0.1.0

version constrains –examples3

6- Caret Version Range (^)

The ^ operator will always **allow non-breaking updates.**

^ allows updates that do NOT change the left-most non-zero version number.

- ^1.2.3
  is equivalent to >=1.2.3 <2.0.0

  Because **major version is 0**, which indicates **unstable API**, so **minor version becomes the breaking boundary**.
- ^0.3 as >=0.3.0 <0.4.0
- ^0.0.3 as >=0.0.3 <0.0.4

caret behavior is different for 0.x versions, for which it will only match patch versions.

version constrains –cont2

- ^1.2.3

Allowed: >=1.2.3 <2.0.0

1.2.3
1.2.4
1.3.0
1.9.9
2.0.0 ❌

- ^0.5.2

>=0.5.2 <0.6.0

why ? Because **major version is 0**, which indicates **unstable API**, so **minor version becomes the breaking boundary**.

Allowed versions
0.5.2
0.5.3
0.5.9
0.6.0 ❌

Composer version constrains –cont3

Example with Patch Level

^0.0.4

Allowed:

>=0.0.4 <0.0.5

Version	Allowed Range
`^1.2.3`	`>=1.2.3 <2.0.0`
`^1.2`	`>=1.2.0 <2.0.0`
`^1`	`>=1.0.0 <2.0.0`
`^0.5.2`	`>=0.5.2 <0.6.0`
`^0.0.4`	`>=0.0.4 <0.0.5`

^ Difference vs Tilde (~)

Operator	Meaning
`^1.2.3`	allow minor + patch
`~1.2.3`	allow patch only

```
^1.2.3 → <2.0.0
~1.2.3 → <1.3.0
```

Simple rule to remember

^ = safe upgrades (no breaking changes)

Stability Constraints

Constraint	Internally
`1.2.3`	`=1.2.3.0-stable`
`>1.2`	`>1.2.0.0-stable`
`>=1.2`	`>=1.2.0.0-dev`
`>=1.2-stable`	`>=1.2.0.0-stable`
`<1.3`	`<1.3.0.0-dev`

Summary

```
"require": {
    "vendor/package": "1.3.2", // exactly 1.3.2

    // >, <, >=, <= | specify upper / lower bounds
    "vendor/package": ">=1.3.2", // anything above or equal to 1.3.2
    "vendor/package": "<1.3.2", // anything below 1.3.2

    // * | wildcard
    "vendor/package": "1.3.*", // >=1.3.0 <1.4.0

    // ~ | allows last digit specified to go up
    "vendor/package": "~1.3.2", // >=1.3.2 <1.4.0
    "vendor/package": "~1.3", // >=1.3.0 <2.0.0

    // ^ | doesn't allow breaking changes (major version fixed - following semver)
    "vendor/package": "^1.3.2", // >=1.3.2 <2.0.0
    "vendor/package": "^0.3.2", // >=0.3.2 <0.4.0 // except if major version is 0
}
```

Comparison

Version Spec	Range	What Changes Allowed
`~1.2.3`	`>=1.2.3 <1.3.0`	patch only
`^1.2.3`	`>=1.2.3 <2.0.0`	minor + patch
`~0.5.2`	`>=0.5.2 <0.6.0`	patch only
`^0.5.2`	`>=0.5.2 <0.6.0`	same as ~

Data Modeling

- Designing the **blueprint of your data**, before you build the database.

1. Conceptual Data Model (High-Level)

2. Logical Data Model

Includes: Attributes (fields) , Relationships, Keys

Core Contepts

1- Entities
Objects or things:
User , Order, Invoice

2. Attributes
Properties of entities:
User → name, email
Order → amount, date

3. Relationships
How entities connect:
One-to-One
One-to-Many
Many-to-Many
One doctor → many patients

Types of Databases & Modeling Styles

1. Relational (SQL)

- Tables, rows, columns
- Strong structure

Examples:

- MySQL
- PostgreSQL

2. NoSQL

- More flexible schemas:

```json
{
  "name": "John",
  "appointments": [...]
}
```

Normalization

- Process of reducing redundancy.

Bad:

```
code

Patient + Doctor name repeated in every appointment
```

Good:

- Separate tables
- Use IDs (foreign keys)

As an architect/product builder:

Normalize → ✅ Clean, scalable

Denormalize → ✅ Faster reads

Two Main Approaches

A. MongoDB + Mongoose (Schema-based NoSQL)

Best when:
- Flexible structure
- Fast development
- Nested data

Example: Patient Model

```js
const mongoose = require('mongoose');

const patientSchema = new mongoose.Schema({
  name: { type: String, required: true },
  dateOfBirth: Date,
  phone: String,
  appointments: [{
    date: Date,
    doctorName: String
  }]
});

module.exports = mongoose.model('Patient', patientSchema);
```

Key Concepts:
- Schema defines structure
- Supports nested documents
- No joins → use embedding or references

Two Main Approaches – cont2

B. SQL + ORM (Sequelize / Prisma)

Best when:
- Strong relationships
- Transactions (finance, healthcare)
- Structured data

Example: Sequelize Model

```js
const { DataTypes } = require('sequelize');
const sequelize = require('../config/db');

const Patient = sequelize.define('Patient', {
  name: DataTypes.STRING,
  dateOfBirth: DataTypes.DATE,
  phone: DataTypes.STRING
});

module.exports = Patient;
```

Querying

- **Specifying attributes for SELECT queries:**

```
SELECT foo, bar FROM ...
```

```
Model.findAll({
  attributes: ['foo', 'bar'],
});
```

You can use sequelize.fn to do aggregations:

```
Model.findAll({
  attributes: ['foo', [sequelize.fn('COUNT', sequelize.col('hats')), 'n_hats'], 'bar'],
});
```

```
SELECT foo, COUNT(hats) AS n_hats, bar FROM ...
```

Applying WHERE clauses

- The Basics

```
Post.findAll({
  where: {
    authorId: 2,
  },
});
// SELECT * FROM post WHERE authorId = 2;
```

Same as

```
const { Op } = require('sequelize');
Post.findAll({
  where: {
    authorId: {
      [Op.eq]: 2,
    },
  },
});
// SELECT * FROM post WHERE authorId = 2;
```

```
Post.findAll({
  where: {
    authorId: 12,
    status: 'active',
  },
});
// SELECT * FROM post WHERE authorId = 12 AND status = 'activ
```

WHERE clauses - cont

```
const { Op } = require('sequelize');
Post.findAll({
  where: {
    [Op.and]: [{ authorId: 12 }, { status: 'active' }],
  },
});
// SELECT * FROM post WHERE authorId = 12 AND status = 'active
```

```
const { Op } = require('sequelize');
Post.findAll({
  where: {
    [Op.or]: [{ authorId: 12 }, { authorId: 13 }],
  },
});
// SELECT * FROM post WHERE authorId = 12 OR authorId = 13;
```

```
const { Op } = require('sequelize');
Post.destroy({
  where: {
    authorId: {
      [Op.or]: [12, 13],
    },
  },
});
```

WHERE clauses – (operators)

```
// Basics
[Op.eq]: 3,                              // = 3
[Op.ne]: 20,                             // != 20
[Op.is]: null,                           // IS NULL
[Op.not]: true,                          // IS NOT TRUE
[Op.or]: [5, 6],                         // (someAttribute = 5) OR (someAttribute = 6)

// Using dialect specific column identifiers (PG in the following example):
[Op.col]: 'user.organization_id',        // = "user"."organization_id"

// Number comparisons
[Op.gt]: 6,                              // > 6
[Op.gte]: 6,                             // >= 6
[Op.lt]: 10,                             // < 10
[Op.lte]: 10,                            // <= 10
[Op.between]: [6, 10],                   // BETWEEN 6 AND 10
[Op.notBetween]: [11, 15],               // NOT BETWEEN 11 AND 15
```

WHERE clauses – (operators)2

```
[Op.all]: sequelize.literal('SELECT 1'), // > ALL (SELECT 1)

[Op.in]: [1, 2],                         // IN [1, 2]
[Op.notIn]: [1, 2],                      // NOT IN [1, 2]

[Op.like]: '%hat',                       // LIKE '%hat'
[Op.notLike]: '%hat',                    // NOT LIKE '%hat'
[Op.startsWith]: 'hat',                  // LIKE 'hat%'
[Op.endsWith]: 'hat',                    // LIKE '%hat'
[Op.substring]: 'hat',                   // LIKE '%hat%'
[Op.iLike]: '%hat',                      // ILIKE '%hat' (case insensitive) (PG only)
[Op.notILike]: '%hat',                   // NOT ILIKE '%hat'  (PG only)
[Op.regexp]: '^[h|a|t]',                 // REGEXP/~ '^[h|a|t]' (MySQL/PG only)
[Op.notRegexp]: '^[h|a|t]',              // NOT REGEXP/!~ '^[h|a|t]' (MySQL/PG only)
[Op.iRegexp]: '^[h|a|t]',                // ~* '^[h|a|t]' (PG only)
[Op.notIRegexp]: '^[h|a|t]',             // !~* '^[h|a|t]' (PG only)

[Op.any]: [2, 3],                        // ANY (ARRAY[2, 3]::INTEGER[]) (PG only)
[Op.match]: Sequelize.fn('to_tsquery', 'fat & rat') // match text search for strings 'fat' and
```

finders

- findAll() ; (get all records)

- findByPk():

```
const project = await Project.findByPk(123);
if (project === null) {
  console.log('Not found!');
} else {
  console.log(project instanceof Project); // true
  // Its primary key is 123
}
```

- findOne()

```
const project = await Project.findOne({ where: { title: 'My Title' } });
if (project === null) {
  console.log('Not found!');
} else {
  console.log(project instanceof Project); // true
  console.log(project.title); // 'My Title'
}
```

Direct link to findone

Raw Queries

```
const [results, metadata] = await sequelize.query('UPDATE users SET y = 42 WHERE x = 12');
// Results will be an empty array and metadata will contain the number of affected rows.
```

Associations

Creating the standard relationships

The Sequelize associations are defined in pairs. In summary:

- To create a **One-To-One** relationship,
  the hasOne and belongsTo associations are used together;
- To create a **One-To-Many** relationship,
  the hasMany and belongsTo associations are used together;
- To create a **Many-To-Many** relationship,
  two belongsToMany calls are used together.

Associations – cont1

- One to One Relationship

```
Foo.hasOne(Bar);
Bar.belongsTo(Foo);
```

```
CREATE TABLE IF NOT EXISTS "foos" (
  /* ... */
);
CREATE TABLE IF NOT EXISTS "bars" (
  /* ... */
  "fooId" INTEGER REFERENCES "foos" ("id") ON DELETE SET NULL ON UPDATE CASCADE
  /* ... */
);
```

Customizing the Foreign Key

```
// Option 1
Foo.hasOne(Bar, {
  foreignKey: 'myFooId',
});
Bar.belongsTo(Foo);
```

Associations – cont2

- **One-To-Many relationships**

```
Team.hasMany(Player);
Player.belongsTo(Team);
```

If I want to pass Options (to change FK for example).

```
Team.hasMany(Player, {
  foreignKey: 'clubId',
});
Player.belongsTo(Team);
```

Associations – cont2

- **Many-To-Many relationships**

```
const Movie = sequelize.define('Movie', { name: DataTypes.STRING });
const Actor = sequelize.define('Actor', { name: DataTypes.STRING });
Movie.belongsToMany(Actor, { through: 'ActorMovies' });
Actor.belongsToMany(Movie, { through: 'ActorMovies' });
```

Design Decisions

- **Embedding vs Referencing (MongoDB)**

Embedding (Denormalized)

```json
Patient {
  name: "John",
  appointments: [...]
}
```

Referencing (Normalized)

```js
Appointment {
  patientId: ObjectId
}
```

Folder Structure (Clean Architecture)

code

```
/models
  patient.model.js
  doctor.model.js

/services
  patient.service.js

/controllers
  patient.controller.js
```

MongoDB

- Mongoose Example

Authentication

- **What is API Authentication?**

  API Authentication is the process of verifying the identity of a client (user, app, or system) trying to access an API.

It answers:

- **Who are you?**
- **Are you allowed to access this resource?**

Why API Authentication Matters

- Protects sensitive data
- Prevents unauthorized access
- Enables user-specific operations
- Supports rate limiting and tracking

Common API Authentication Methods

1. API Key Authentication
2. Basic Authentication
3. Bearer Token (JWT) Authentication
4. OAuth 2.0
5. HMAC Authentication

Authentication- Key Authentication

- A simple token (API Key) is passed with each request.

Request:

```http
GET /api/v1/users
Host: example.com
Authorization: ApiKey 12345abcde
```

Or as query param:

```http
GET https://example.com/api/v1/users?api_key=12345abcde
```

Authentication- Key Authentication-cont

Pros

- Easy to implement
- Lightweight

Cons

- Less secure (no user identity)
- Easily exposed if not protected

Authentication- Basic Authentication

Uses username and password encoded in Base64.

📌 **Example**

```http
Authorization: Basic dXNlcm5hbWU6cGFzc3dvcmQ=
```

(Base64 of username:password)

Authentication- Basic Authentication - cont

Pros

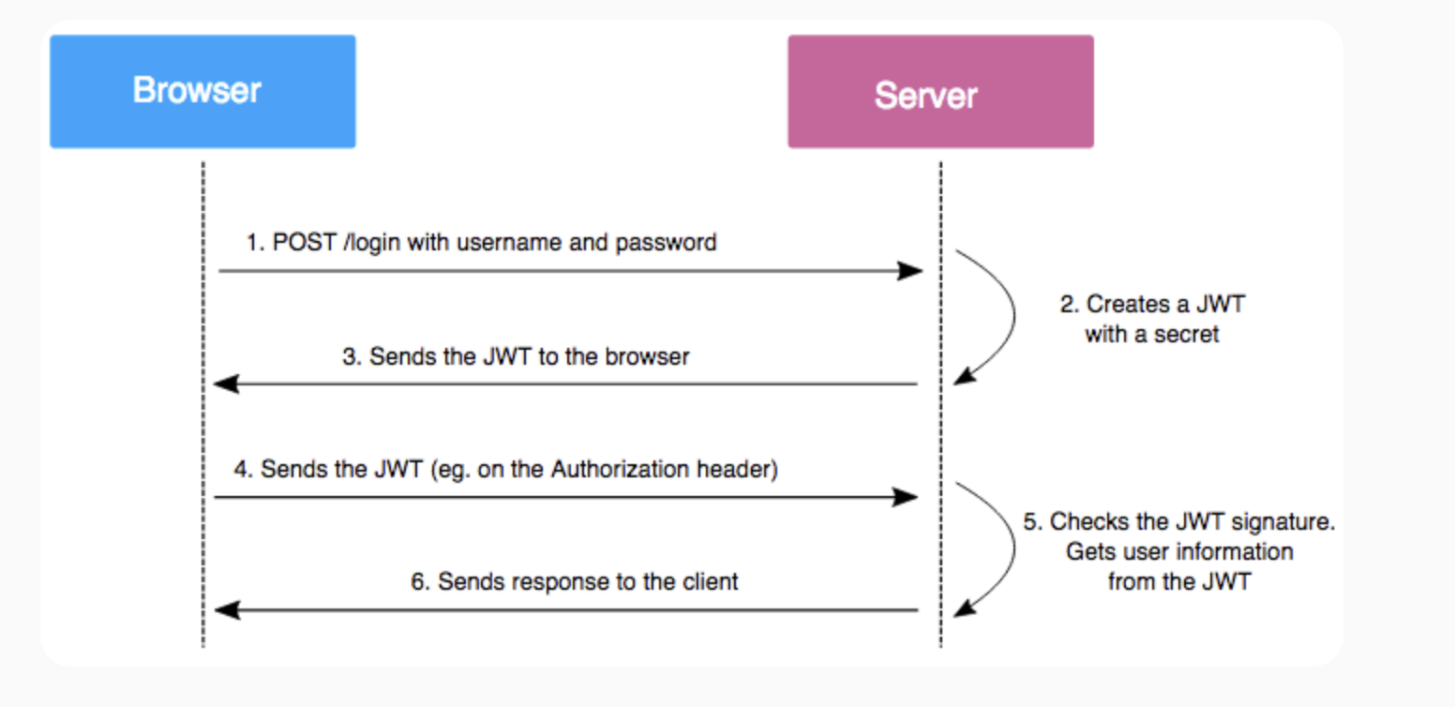
- Very simple

Cons

- Not secure without HTTPS
- Credentials sent on every request

Bearer Token (JWT) Authentication

Client logs in → receives a token → uses it for future requests.



Bearer Token (JWT) Authentication

Login Response:

```json
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

API Request:

```http
GET /api/v1/profile
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

Bearer Token (JWT) Authentication

Pros

- Stateless (no DB lookup)
- Scalable
- Secure (if implemented correctly)
-

Cons

- Cannot easily revoke tokens
- Requires careful expiration handling

OAuth 2.0

- **Concept**

Delegated authorization (login via third-party like Google).

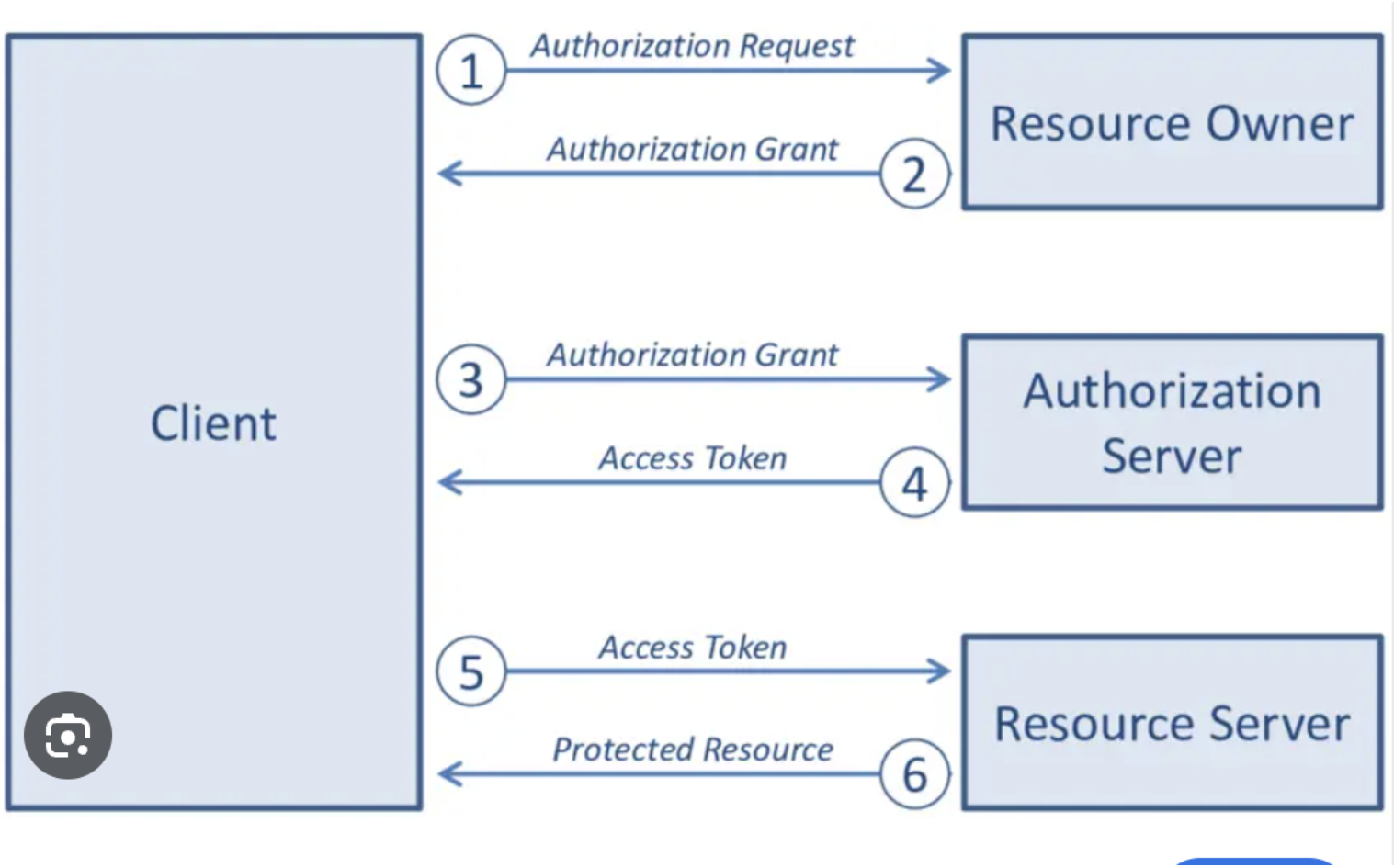
- Used by platforms like: Social logins
- Enterprise integrations

OAuth 2.0

Flow (Simplified)

- User is redirected to authorization server
- User logs in & approves access
- Client receives **authorization code**
- Exchanges it for **access token**

OAuth 2.0


Client
1
Authorization Request
Resource Owner
Authorization Grant
2
3
Authorization Grant
Authorization Server
Access Token
4
5
Access Token
Resource Server
Protected Resource
6

Oauth Grant Types

1- Authorization code (google, facebook …)
send username ,password , get auth code , then exchange with access token , then use this token for resource access .

2- Implicit.
We use a username and password to get an access token directly. (token is given from 3rd Party (Google).

3- Resource owner password credentials.

(here we do trust the client for our username and password.)

4- Client credentials (M2M)
require client-id , secret to communicate (used when integrating between systems).